

metastable-3: status

What this branch is

metastable-3 starts from main at 90d9d26736. The earlier metastable-2 branch (tip 87e0853aff, pushed to the fork) is abandoned; its code may be read and copied but nothing here builds on it. The goal is a deterministic checker that, pointed at a Hydro program, decides whether the schedule alone can make it do work its input did not require, and names where in the program that hazard arises. The first deliverable, now complete, is a corpus of labeled witness programs that the checker will be judged against.

Commits so far

```
177f3d259f feat(hydro_test): witnesses W6, W8, W10, W11, W12: heartbeat and TC controls, rebalancing, c
93a970ae9b docs(design): verdict rule follows the mechanism-based labels
484fd4e109 fix(hydro_test,design): label witnesses by mechanism, not by outcome under one harness
086c9bfdfb feat(hydro_test): witnesses W3, W4, W5: gossip resend-until-ack, backoff retry, election star
bc5c42c6b6 docs(design): the checker searches schedules under fixed input; twins are incidental
4b07732d63 docs(design): what carries over from metastable-2 under the stricter constraints
60abaf0a28 feat(hydro_test): witnesses W2, W7, W9: crdt_gossip load harness, cache with expiry, bounded
9953dba28e chore(hydro_test): add empty witnesses module for the corpus
7deaa01d9b docs(design): witness corpus specification
23a5014705 feat(hydro_lang,hydro_test): port deterministic prompt-schedule runner and rpc_retry witness
```

The corpus

Twelve programs and twenty-two labeled configurations live in `hydro_test/src/cluster/witnesses/` and `hydro_test/src/cluster/rpc_retry.rs`. The full table with headline numbers and confirming test names is in `witnesses/mod.rs`. A configuration is hazardous when some schedule makes the same input cost more work, hazardous, mitigated when a parameter keeps that from collapsing the program under load, and benign when work is bounded by the input under every schedule.

file	configuration	basis
<code>rpc_retry</code>	<code>max_attempts = 3 / 1</code>	hazardous, by exhibited collapse / benign
<code>backoff_retry</code>	<code>backoff = true / false</code>	no ground truth, expected hazardous / benign
<code>bounded_queue_rejection</code>	<code>max_backlog = Some(100) / None</code>	no ground truth, expected hazardous / benign
<code>cache_thundering_herd</code>	no coalescing, request dated / fill dated; coalescing	hazardous, by exhibited collapse / no ground truth
<code>compaction_falls_behind</code>	<code>compaction_reserve = 0 / 8</code>	hazardous, by exhibited collapse / no ground truth
<code>crdt_gossip_load</code>	<code>g_set_gossip</code> unmodified	benign, by assurance argument
<code>election_stampede</code>	<code>timeout_spread_ticks = 0 / 3</code>	hazardous, by exhibited collapse (both)
<code>gossip_resend</code>	<code>ack_timeout_ticks > 0 / = 0</code>	hazardous, by exhibited collapse / benign
<code>lease_renewal_storm</code>	<code>resend_after_ticks = Some(4) / None</code>	hazardous, by exhibited collapse / benign
<code>pure_heartbeat_load</code>	<code>pure_heartbeat</code> unmodified	benign, by assurance argument
<code>rebalancing_ping_pong</code>	<code>cooldown_ticks = 0 / 8</code>	hazardous, by exhibited collapse / no ground truth
<code>transitive_closure_load</code>	<code>productive_tc</code> unmodified	benign, by assurance argument

All 45 corpus tests pass together (`cargo test -p hydro_test --lib witnesses::`, 92 s of test time, about 5 minutes wall clock including the build), and the 3 `rpc_retry::sim_tests` pass in 8 s. Every program takes time as a stream parameter, keeps its state in Hydro operators, and was labeled by a hand-computed expectation written before measurement.

Guiding documents

`design_docs/2026-09_witness_corpus_spec.md` defines the labels and the protocol every witness follows, and was written so that an author could work from it without knowing anything about the checker.

`design_docs/2026-09_what_carries_over.md` reviews every technique from `metastable-2` against the new constraints and ends with the ordered experiments below.

Next

The checker work starts with section 5 of `2026-09_what_carries_over.md`. Every experiment searches schedules under the witness's baseline workload with no load trigger, and its pass criterion is agreement with the corpus labels.

1. E1: port `edge_counts`, `hold_schedule` and `with_empty_ticks_allowed`, and show that a held response edge on `rpc_retry` produces excess sends while `max_attempts = 1` sends exactly the input. Nothing later is trusted until this passes.
2. E2: hold responses on a delay grid and show the excess rises with delay above the timeout, with `backoff_retry` rising more slowly.
3. E3: release-size moves on the arrivals edge, to reach server-side bounds in `bounded_queue_rejection`.
4. E4: the goal-free sweep over every labeled configuration, judged by the rule that a rising excess on some edge is hazardous and a flat one is benign.
5. E5 and E6: the static identity pass for localization candidates, and the payload-shape column on `election_stampede`.
6. E7: localization by negative leaves and forks on the cells E4 finds.

Open bug, out of scope

`transitive_closure_load` found that the unmodified `hydro_test/src/local/productive_tc.rs` yields an incomplete closure under 212 of 256 fuzzed schedules. The program replaces its frontier with each tick's novel facts, so a tick that admits a graph without a step token discards the pending frontier. Completeness of that program depends on the schedule. It is recorded here and left alone.

First checker experiment: random schedules on `rpc_retry`, no simulator changes

`hydro_test/src/cluster/witnesses/checker_experiments.rs` asks whether `flow.sim().fuzz(..)` alone, on the baseline workload with no trigger, recovers the label of `rpc_retry`. It does: over 512 random schedules with a timeout of 2 ticks and six clock elements per round, every schedule at three attempts cost more than the 48 distinct requests (mean 25 extra sends, max 55, and the server redid exactly that work), and no schedule at one attempt did, although the delays the schedules imposed were the same (1 to 6 ticks). Raising the timeout to 4 and 6 shows extra work rising with the observed delay within a run (4.8 to 8.1 at timeout 4) and falling across timeouts (25.1, 7.7, 2.3). Each 512-schedule run takes about 10 s once built. Two limits: the timeout had to be small for a uniform random draw to reach it, so the checker will need to direct the search or meter time; and a passing schedule's bytes are not exposed by `fuzz`, so although the per-tick trace names the batch decision that held a reply, it can only be replayed for schedules the fuzzer saved.

Second checker experiment: hold one edge, across the whole corpus

The first experiment's limit was that a uniform random draw will not hold a reply through forty consecutive clock releases. `hydro_lang/src/sim/hold_one_hook.rs` answers it with a bolero driver that follows the prompt schedule everywhere except at one named `use::batch` hook, which releases nothing while the harness says so, so a delay of any length on one edge is a single decision. Two small changes to the simulator were needed, both inert unless a hold is set: hooks tell the driver who is asking (a thread-local the scheduler sets around each decision, with the hook named `location#index-within-tick [item type]` because every batch in one `sliced!` block reports the block's location), and `SimTick::can_run` ignores a held hook, so a tick whose only loaded hook is held parks instead of being forced to release. Without the second change every hold was refused after one round, on every tick and not only on timerless ones.

The `hold_sweep` tests in `checker_experiments.rs` run every one of the 22 labeled configurations under its own baseline workload with no trigger, discover the hooks that ever have input, hold each from round 20 for

0, 5, 10, 20, 40, 60, 80 and 100 rounds, and read extra work from the witness's own counters. The verdict is hazardous if some hold adds work over the unheld run and benign if none does; the hook with the largest gain is the localization. Twenty-one of twenty-two verdicts agree with the labels, no hold was refused, and the whole run takes about six minutes with the tests in parallel. On `rpc_retry` at its real timeout of 40 the responses hook reproduces the hand computation exactly (0 0 0 0 80 160 320) and the server's arrivals hook has the largest gain (0 0 0 0 4 136 498 588); at one attempt every curve is zero. The same two edges localize every request and response program in the corpus. The disagreement is `compaction_falls_behind` with `compaction_reserve = 8`, labeled "hazardous, mitigated" and found benign: with the reserve, neither a held clock nor a held operation stream adds any work, because the reserve compacts the segment before any get is costed; the hazard the label describes needs offered load above capacity, which no schedule of a fixed input produces. Whether that configuration should carry the label is a question about the definition rather than the measurement. A second observation for the definition: the two rebalancing configurations are found hazardous through the same hook with nearly the same curve, so the sweep sees "a delayed dump of tasks causes migrations" and does not separate the ping-pong the cooldown removes from the legitimate moves it leaves.

Relabeling: every label now carries its basis

The owner ruled that a label is ground truth on one of two bases only. A configuration is hazardous, by exhibited collapse, when a test drove it into collapse, which is a constructive proof. A configuration is benign, by assurance argument, when no collapse was found and the author wrote an argument that work is a function of the input alone under every schedule, either a closed form checked under random schedules or the structural absence of any operator that can emit a second send; the owner accepts such arguments as the best basis a negative label can have, since failing to collapse a program is not evidence that it is robust. The qualifier "mitigated" was removed everywhere, because a tuning parameter such as backoff, a queue bound, a compaction reserve or a cooldown cannot remove a mechanism; whether it suffices depends on a workload unknown when the program is written, so it only moves the threshold. The five configurations that carried the qualifier have no ground truth and are recorded as "no ground truth; tool expected to find amplification". The language of matched pairs was removed from the spec and the report, since it rested on the same fallacy. The counts are ten ground-truth hazardous, seven ground-truth benign, and five expectation-only; the checker agrees with ground truth on 17 of 17 and with the expectations on 4 of 5, the miss being compaction with a reserve of 8, which is now recorded as a checker miss on a program that has the mechanism rather than as a question about the definition. Whether the checker may vary the composition of its input to reach a threshold is left open. The owner has ruled that a storm lasting hundreds of rounds after a short trigger is an exhibited collapse, so the election configurations and rebalancing with no cooldown, whose storms ended before their runs did, are ground-truth hazardous.

Third checker experiment: the verdict from counts the simulator collects itself

The second experiment still read work from counters the witness authors exposed, which a tool pointed at an arbitrary crate cannot assume. `hydro_lang/src/sim/work_counts.rs` replaces them with counts the simulator collects passively while a run executes and hands to the harness afterwards: records released by every `use::batch` hook (counted on the host side of the dylib boundary in `run_hooks`, through a new `SimHook::pending_release_count`), network messages (the deploy path's `_network_metrics()` pass-through placed in the simulator's send pipelines and read from DFIR metrics), DFIR handoff items, and DFIR subgraph runs, each broken down by hook, sending location and cluster member. Collection is off unless a harness enables it, no scheduling decision changed, and the simulator's own 165 sim tests pass. The `generic_measure` tests in `checker_experiments.rs` rerun all 22 hold sweeps with the same inputs and grid, reading nothing from the programs, in 377 s of test time against 354 s before, so counting costs about six percent.

The reasoning written before measuring held: hook releases and network messages cannot be raised by a delay unless the program derives new records, because over a run each is what arrived minus what is still buffered; handoff items can, because they include state re-read every tick, and they duly called four of the seven benign configurations hazardous (the outstanding table, the waiting lookups, the inbox and the renewal

table are all re-read more often while held). Network messages reproduce the `rpc_retry` hand count exactly (gain 588 at the arrivals hook), since sends plus serves are the messages in both directions.

The measurement then forced one refinement the reasoning had not made. A plain total of admitted records called `election spread=3` benign, because the vote requests the followers add are outnumbered by the heartbeats the leader stops sending while the cluster is leaderless, and the total falls from 960 to 944. The finest breakdown, every hook on every member, caught that but called `rpc_retry` at one attempt hazardous, because a request whose reply is held past the timeout is abandoned rather than completed, and the `abandoned` hook admits 202 more records while `completed` admits 202 fewer. What separates the two cases is what competes for capacity: a message to another location consumes that location's capacity and is read per sender, where a quiet leader cannot hide a busy follower; records a location admits into its own ticks consume its own capacity and are read in total, where a substitution nets to zero. The rule is: hazardous if some hold raises any member's outgoing messages or the program's total admitted records; benign otherwise. Under it, 21 of 22 verdicts agree with the hand-counted verdicts and name the same held hook, the exception in naming being `election spread=0`, where the rule's largest gain is at the timer hook one line from the `Msg` inbox the hand count named, both of which rise.

The one disagreement with the hand count is `compaction_falls_behind` with `compaction_reserve = 0`, as predicted. Its work is a cost field in a payload and a `u64` segment length, so the operations and clock elements it admits are the same under every schedule (flat at 1440) while the hand count rises by 9979 units. Handoff items do see the collapse, through the backlog the store re-chains through `cross_singleton` and `scan` every tick (6478 to 170938), but they cannot tell that re-reading from the benign re-reading above, so they are not usable for the verdict. A witness that materialized its scan as records would be visible to the rule; this one was left unmodified. Against the labels, the generic rule agrees with 16 of the 17 ground-truth labels (compaction reserve 0, an exhibited collapse, is the miss) and with 4 of the 5 expectation-only configurations (compaction reserve 8 is the miss, as before), against 17 of 17 and 4 of 5 for the hand-counted sweep. What remains supplied by hand is the workload and the per-program input wiring.

The compaction witness is excluded from the scorecard

On review, the owner ruled that the tool is right about `compaction_falls_behind` and the witness is a poor one. Its reads do not scan anything: the program computes each read's cost as an integer (one plus the segment length) and subtracts it from an integer budget, so no records move because no work is done. The hand-chosen counter saw a rise because it read the modeled number; the tool saw 1440 admitted records at every hold length because that is what happened. Both compaction rows are now marked in the corpus table as work modeled by a counter rather than performed, and they are excluded from the checker scorecard until someone rewrites the witness so that a read joins against the uncompact segment and produces one record per record scanned. Without those two rows the generic measure agrees with the hand count on 20 of 20 configurations, with ground truth on 16 of 16, and with the expectations on 4 of 4.

Noted, not built

The tool counts records at two places, batch-hook releases and network messages, and does not count records flowing between operators inside a tick. A witness whose redundant work is a real scan inside a tick, for example a read that joins against an uncompact segment and produces one record per record scanned, would be invisible to the tool as it stands, because the scan's output never crosses a hook or the network. The one intra-tick count tried, DFIR handoff items, also counts the replay of persisted state into each tick, and on that count four of the seven benign configurations read hazardous. Separating replay from newly produced records would require an operator-level count built from DFIR's operator graph. The owner decided not to build that until an existing witness justifies it, and no existing witness does: the only candidate, the compaction store, models its scan rather than performing it.

A question underneath that decision remains open. State re-read while a tick waits (the outstanding table replayed and its timeouts re-checked on every tick during a hold) is real CPU, proportional to state size times delay, with no feedback. The assurance arguments for the benign labels treat it as the cost of waiting rather

than as amplification. If it were counted as amplification, several assurance arguments would be wrong and the definition would need a feedback clause. The owner has not ruled.

The checker as a library function

`hydro_lang/src/sim/amplification.rs` packages the third experiment as one call. `check(flow, &config, async |round| { ... })` takes a `SimFlow`, a `CheckConfig` (rounds, the round at which holds begin, and the hold grid; `CheckConfig::new(240)` gives the corpus defaults), and a closure that sends one round of the fixed workload, after which the checker awaits `quiesce()` itself. It compiles the simulation once, runs the discovery pass under the prompt schedule, holds each discovered hook for each grid length, and returns a `Report` carrying the verdict, one curve per hook (extra admitted records and extra per-sender messages at each hold length, the full counts, and any refused holds), and, when hazardous, a `Location` naming the held hook, its `use::batch` source location, what rose, the largest gain, the hold length at which a gain first appeared, and the hooks that admitted more records under that hold. The verdict rule is unchanged from the third experiment. Nothing is read from the program's outputs, and no simulator scheduling changed; the module is additive and the simulator's own tests pass.

The `library_check` tests in `checker_experiments.rs` wire all 22 corpus configurations exactly as the earlier harnesses do and assert the function returns the verdict and held hook that the generic-measure sweep recorded. All 22 pass, and every gain matches to the unit. The 22 tests take 44 s of wall clock together, against 377 s for the generic-measure sweep, because the simulation is compiled once per configuration rather than once per run. What the caller still supplies is the workload and the wiring of `sim_input` streams to the program's parameters.

Snapshot hooks are now held too

Until now the checker held only `use::batch` hooks, and the earlier status entries said `use::snapshot` hooks were neither held nor counted. That gap is closed. A `use::snapshot` is the other decision the simulator makes inside a tick: which queued version of a singleton the tick observes, or whether it observes the last one again. The checker now discovers snapshot hooks (they carry the word `snapshot` in their identity) and holds each one in turn like a batch. A held snapshot keeps its tick observing the version it saw last before the hold began, for `k` rounds, so the tick acts on stale state, which is what a delayed state update looks like from inside the program. When the hold ends the hook skips to the newest queued version, the analogue of a batch hook releasing its whole buffer, so that the perturbation is a delay of `k` rounds rather than a lag that would otherwise persist for the rest of the run (the prompt policy observes queued versions one per tick, oldest first). Snapshot releases are still not counted toward work, because a snapshot is one value per tick execution and not a record the program produced; the work a stale snapshot provokes shows up where it always does, in the records and messages the program then derives. The simulator change is metadata (singleton hooks report a location and a kind) plus the scheduler telling the driver whether it is forcing a decision, and a small skip-to-newest rule in the driver; nothing changes unless a hold is set. The simulator's own tests pass, and three new driver unit tests pin down the hold, forced, and catch-up behaviour.

The corpus contains three snapshot hooks that ever make a decision: `rpc_retry.rs:330`, the server metrics tick's snapshot of its backlog depth; `crdt_gossip.rs:70`, the G-Set snapshot each member broadcasts on every pump; and the `crdt_gossip` harness's own observer. Holding each at every grid length gave a gain of exactly zero on every count in both `rpc_retry` configurations and in `crdt_gossip`; the two program snapshots were confirmed pinned at every length, and the observer's tick, having no other input, parked. The reasons are mechanical: the backlog-depth snapshot feeds a metrics accumulator nothing acts on, and the gossip snapshot is broadcast once per pump element whatever its contents, so staleness changes what a message carries and not how many there are. No verdict and no named location changed on any of the 22 configurations, and no benign configuration became hazardous. The 22 library tests now take 63 s of wall clock against 44 s before, the difference being the hold runs for the three snapshot hooks. Every corpus program keeps the state its sends depend on in `use::state` inside the tick, where the scheduler makes no decision about it, so nothing in the corpus can be provoked by a stale snapshot; a witness that judges a retry against a snapshot clock, or issues a fetch because a snapshot cache still shows a miss, would be the

first test of this extension that could fail.

Blind test protocol

The corpus so far was written by authors who were instructed to ignore the checker but who inherited this conversation's context, so the firewall between witness authors and the checker was by instruction rather than by construction. The next round of witnesses is written under a construction-level firewall. The owner opens a fresh session with no history and hands it only `design_docs/2026-09_blind_witness_task.md` and the repository; that brief is self-contained, names the files the author may not read, and describes neither the checker nor any method of detection. The author writes and labels new programs under `hydro_test/src/cluster/witnesses/blind/`, each label resting on an exhibited collapse or an assurance argument, and commits them. The checker is then run on those programs by `hydro_lang::sim::amplification::check`, wired by an operator who has not read the programs first and who reads only their public signatures and outputs to supply the round closure. The verdicts and named locations are compared to the labels, and every disagreement is explained mechanically before anything about the checker is changed. `design_docs/2026-09_witness_corpus_spec.md` has been cleaned of every reference to the checker's method for the same reason, so that it too can be handed to a context-free author.

The hold grid is gone; holds escalate to the horizon

The owner pointed out that the fixed grid of hold lengths (0, 5, 10, 20, 40, 60, 80, 100) and the hold start at round 20 were chosen to fit the corpus, whose timeouts are all 40 ticks or less, and that a user cannot choose a grid without knowing the program's timescales. `CheckConfig` now has a single setting, the run length in rounds (the horizon, default 1000), and every hold begins at round 1. For each decision point the checker holds for 1, 2, 4, 8 and so on, doubling, up to and including a hold that lasts to the end of the run, and it runs every length for every point so the report carries the complete curve; the shape of the curve is information for the reader and plays no part in the verdict. A benign verdict is printed as "no reaction to a delay of up to N rounds was found at any decision point", with N the horizon. The checker prints one line per decision point as its holds complete, so a long run shows its findings as it goes.

The location rule changed with it. The last hold in the sequence is a permanent partition of one edge, and under a permanent hold on any edge of a resend loop the sender exhausts its resends on every request, so several edges of the same loop reach one ceiling on sender messages and the largest gain no longer distinguishes them (in `batch_flush` three hooks tied at 57348 extra sends at a horizon of 240, in `two_phase_commit` three tied at 13564, in `visibility_queue` three at 9282). The checker now ranks decision points by the shortest hold length at which extra work first appears, then by the extra work at that length, and only then by the largest gain over the curve.

All 29 verdicts (22 corpus, 7 blind) are unchanged at both a horizon of 240 and the default of 1000. Under the new rule 13 of the 17 hazardous configurations name the same decision point the fixed grid named, including every request/response program (the server's arrivals hook, which reacts first and hardest, at a hold of 64 rounds against the 40-tick timeout), lease renewal (`Ack`), rebalancing (task arrivals), the fill-dated cache (origin clock), election spread 0 (timer), and three of the four blind programs (`batch_flush` sink arrivals, `two_phase_commit` participant timer, `visibility_queue` worker arrivals). Four differ: the request-dated cache names the origin's clock (reacts at a hold of 1 round) instead of the `Fill` hook (2 rounds); election spread 3 names the timer (8 rounds, 3948 extra sends) instead of the `Msg` inbox (16 rounds, and its large reaction sits near 20 rounds, between the sequence's 16 and 32); `gossip_resend` names the members' timer (2 rounds, 9952 extra admitted) instead of `Ack` (4 rounds), because a re-send is judged against the member's own clock so a paused clock makes every outstanding delta look overdue at once; and `log_catchup` names a leader timer whose first reaction is a single extra message at 8 rounds, over the `Fetch` hook's 1759 at 16, which is the rule's weakness made visible: a slight reaction to a short delay outranks a strong reaction to a longer one. In every case the point named is an edge of the loop the mechanism runs through, and the report carries every point's curve.

Cost at the default horizon is dominated by simulating the storms the long holds provoke. Benign programs take 2 to 30 seconds each; hazardous ones take 15 seconds to several minutes, with `batch_flush` at 1340

seconds, `gossip_resend` with re-sends at 505, `two_phase_commit` at 365 and `visibility_queue` at 209. All 29 configurations take 54 minutes of test time, about 23 minutes of wall clock on four threads; at a horizon of 240 they take about two minutes.

Decision points are named by their own line

A report used to name a decision point by the source position of the `sliced!` block it belonged to, followed by an index whose order was not declaration order, so a reader could not find the operator from the name. The cause is in the compiler rather than the checker: the simulator positions operators by a runtime backtrace, and `rustc` collapses the debug-info location of code expanded from an external macro to that macro's call site, so for a program outside `hydro_lang` every `use::batch` and `use::snapshot` inside one `sliced!` block reported the block. The fix records the position while the macro expands, where the span of the user's tokens is exact: a new `span_location!` macro in the `copy_span` crate reads the file, line and column of the `use` expression, `sliced!` passes them into a new field on the operator's backtrace record, and the simulator prefers that field when it is present. Nothing else reads the field. The hook identity is now `file:line`, without the column, and a report shows an index only when two decision points share a line, which happens nowhere in the corpus or the blind set. The `rpc_retry` report now reads `rpc_retry.rs:193, batch of Response<u64>` for the client's reply input and `rpc_retry.rs:296, batch of Request<u64>` for the server's arrivals, and a reader who opens the file at those lines finds the two `use::batch` calls. All 29 validation tests pass at the default horizon with every verdict and named decision point unchanged, and the overview's reports were re-captured from that run.

The checker has a user-facing entry point

Until now the checker was a library function plus hand-written wiring per program, which is not a tool that can be pointed at a Hydro function. It now has one. The attribute `#[amplification_check(...)]`, exported from `hydro_lang::sim::amplification` and implemented in the new proc-macro crate `hydro_amplification_macro`, is placed on a Hydro function and generates a `#[test]` that builds the simulation from the function's signature, feeds a steady workload, runs `check`, prints the report, and records a summary line. The macro classifies each parameter by type: `&Process` and `&Cluster` are locations it creates (a cluster's size is given by name), every `Stream<_, _, Unbounded, ..>` is an input it feeds once per round, and anything else is a plain value that must be given by name; generic type parameters are given as `T = u64`. A stream of `()` is fed the same way as any other, which is what a timer needs, so timers require nothing in the attribute; this also covers the blind programs, several of which use `Stream<()>` as a data input rather than a timer. The `workload(...)` clause is a separate, optional statement of the operating point (one value per round by default, lists allowed so that a verdict's dependence on the rate is visible), and `round = |round, inputs| { ... }` replaces the generated workload for programs whose steady input is not a rate. Returned structs derive `SimOutputs`, a new trait that sinks every stream field; single streams, keyed streams, and tuples need nothing. Data inputs draw values from a new trait `InputValue`, sequential rather than random, chosen over `bolero's TypeGenerator` because that derive is unreachable from user crates. The script `cargo-check-amplification <crate> [substring]` at the repository root runs every generated test and prints a summary table from `target/amplification/results.tsv`.

Every corpus program and every blind program now carries the attribute, twenty-eight configurations in all, and the generated harnesses reproduce the hand-wired verdicts and locations. Two things could not be annotated. The full-state gossip `g_set_gossip` in `hydro_std` returns a `Singleton`, which the harness cannot sink without a step of its own, and `hydro_std` does not enable `hydro_lang's` simulator outside its tests. Everything else fit; the compaction store needed a four-line `InputValue` implementation for its operation type, and the cache, the rebalancing workers, the transitive closure, and the token bucket use `round` closures. The generated harness feeds every stream, including timers the hand harnesses left silent (the two metrics ticks of `rpc_retry`, for instance), so some reports show more decision points and a larger baseline than the hand-wired ones; the extra points show no extra work and no verdict or location changed. The hand-written `library_check` and `blind_check` tests remain as the validation of the library function itself.

The retry-governance set, and the close of this spike

A third set of witnesses, five configurations of one request-response service under `hydro_test/src/cluster/witnesses/blind` differing only in the client's retry policy, was written by a fresh session from the blind brief and checked with the rule frozen. The checker agreed with three of the author's five labels and produced its first false hazardous verdicts, on the success-refilled token budget and the circuit breaker, both labeled benign by assurance argument. The success budget's extra work grows about linearly in the hold length on the server's arrivals edge and stays within the author's bound of 1.25 times the input plus ten, so the definition's two clauses, growth with delay and a bound in terms of input alone, part ways on it; the breaker sheds input while open, which the count does not read. The record is `design_docs/2026-09_blind_budget_results.md`.

This spike is closed on that result. The closeout document, `design_docs/2026-09_spike_closeout_and_next.md`, states what was built, how labels are established, the results with their standing, the open problems in priority order with a first step for each, and the commands that run everything; it is the document the next spike should start from.